

NUMPY FUNCTIONS

```
In [ ]: np.sort
Return a sorted copy of an array.
```

```
In [1]: import numpy as np
a = np.random.randint(1,100,15)
a
```

```
Out[1]: array([31, 61, 94, 22, 87, 25, 26, 28, 17, 23, 11, 95, 37, 39, 13],
             dtype=int32)
```

```
In [2]: b = np.random.randint(1,100,24).reshape(6,4)
b
```

```
Out[2]: array([[71, 27, 77, 60],
               [48, 70, 92, 65],
               [64, 44, 82, 51],
               [17, 40, 99, 94],
               [28, 41, 57, 82],
               [65, 82, 48, 40]], dtype=int32)
```

```
In [3]: np.sort(a)
```

```
Out[3]: array([11, 13, 17, 22, 23, 25, 26, 28, 31, 37, 39, 61, 87, 94, 95],
             dtype=int32)
```

```
In [4]: np.sort(a)[::-1]
```

```
Out[4]: array([95, 94, 87, 61, 39, 37, 31, 28, 26, 25, 23, 22, 17, 13, 11],
             dtype=int32)
```

```
In [5]: np.sort(b)
```

```
Out[5]: array([[27, 60, 71, 77],
               [48, 65, 70, 92],
               [44, 51, 64, 82],
               [17, 40, 94, 99],
               [28, 41, 57, 82],
               [40, 48, 65, 82]], dtype=int32)
```

```
In [9]: np.sort(b,axis = 0) #column
```

```
Out[9]: array([[17, 27, 48, 40],
               [28, 40, 57, 51],
               [48, 41, 77, 60],
               [64, 44, 82, 65],
               [65, 70, 92, 82],
               [71, 82, 99, 94]], dtype=int32)
```

```
In [10]: np.sort(b,axis=1)
```

```
Out[10]: array([[27, 60, 71, 77],
               [48, 65, 70, 92],
               [44, 51, 64, 82],
               [17, 40, 94, 99],
               [28, 41, 57, 82],
               [40, 48, 65, 82]], dtype=int32)
```

```
In [ ]: np.append
The numpy.append() appends values along the mentioned axis at the end of
```

```
In [11]: np.append(a,250)
```

```
Out[11]: array([ 31,  61,  94,  22,  87,  25,  26,  28,  17,  23,  11,  95,  37,
                39,  13, 250])
```

```
In [ ]: np.concatenate
numpy.concatenate() function concatenate a sequence of arrays along an ex
```

```
In [16]: c = np.arange(6).reshape(2,3)
d = np.arange(6,12).reshape(2,3)
print(c)
print(d)
```

```
[[0 1 2]
 [3 4 5]]
[[ 6  7  8]
 [ 9 10 11]]
```

```
In [17]: np.concatenate((c,d),axis = 0)
```

```
Out[17]: array([[ 0,  1,  2],
               [ 3,  4,  5],
               [ 6,  7,  8],
               [ 9, 10, 11]])
```

```
In [18]: np.concatenate((c,d),axis = 1)
```

```
Out[18]: array([[ 0,  1,  2,  6,  7,  8],
               [ 3,  4,  5,  9, 10, 11]])
```

```
In [ ]: np.unique
With the help of np.unique() method, we can get the unique values from an
given as parameter in np.unique() method.
```

```
In [20]: e = np.array([1,1,2,2,2,23,4,4,5,6,7,8,8])
np.unique(e)
```

```
Out[20]: array([ 1,  2,  4,  5,  6,  7,  8, 23])
```

```
In [ ]: np.expand_dims
With the help of Numpy.expand_dims() method, we can get the expanded dime
```

```
In [22]: a.shape
```

```
Out[22]: (15,)
```

```
In [26]: np.expand_dims(a,axis=0).shape
```

Out[26]: (1, 15)

```
In [28]: np.expand_dims(b,axis = 1).shape
```

Out[28]: (6, 1, 4)

```
In [ ]: np.where
        The numpy.where() function returns the indices of elements in an input ar
```

```
In [29]: # find all indices with value greater than 50
        np.where(a>50)
```

Out[29]: (array([1, 2, 4, 11]),)

```
In [30]: # replace all values > 50 with 0
        np.where(a>50,0,a)
```

Out[30]: array([31, 0, 0, 22, 0, 25, 26, 28, 17, 23, 11, 0, 37, 39, 13],
dtype=int32)

```
In [31]: np.where(a%2==0,0,a)
```

Out[31]: array([31, 61, 0, 0, 87, 25, 0, 0, 17, 23, 11, 95, 37, 39, 13],
dtype=int32)

```
In [ ]: np.argmax
        returns maximum value present in the index position
```

```
In [32]: a
```

Out[32]: array([31, 61, 94, 22, 87, 25, 26, 28, 17, 23, 11, 95, 37, 39, 13],
dtype=int32)

```
In [33]: np.argmax(a)
```

Out[33]: np.int64(11)

```
In [34]: np.argmax(b,axis = 0)
```

Out[34]: array([0, 5, 3, 3])

```
In [35]: b
```

Out[35]: array([[71, 27, 77, 60],
[48, 70, 92, 65],
[64, 44, 82, 51],
[17, 40, 99, 94],
[28, 41, 57, 82],
[65, 82, 48, 40]], dtype=int32)

```
In [ ]: #np.argmin
```

```
In [36]: np.argmin(a)
```

Out[36]: np.int64(10)

```
In [ ]: ### np.cumsum
```

numpy.cumsum() function is used when we want to compute the cumulative su

```
In [37]: np.cumsum(a)
```

```
Out[37]: array([ 31,  92, 186, 208, 295, 320, 346, 374, 391, 414, 425, 520, 557,
                596, 609])
```

```
In [39]: np.cumsum(b,axis= 1)
```

```
Out[39]: array([[ 71,  98, 175, 235],
                [ 48, 118, 210, 275],
                [ 64, 108, 190, 241],
                [ 17,  57, 156, 250],
                [ 28,  69, 126, 208],
                [ 65, 147, 195, 235]])
```

```
In [40]: # np.cumprod
np.cumprod(a)
```

```
Out[40]: array([          31,           1891,           177754,
                3910588,          340221156,          8505528900,
                221143751400,        6192025039200,        105264425666400,
                2421081790327200,       26631899693599200,       2530030470891924000,
                1377407054453429920, -1621357097444887968, -263089819307399196
                8])
```

```
In [ ]: ### np.percentile
```

numpy.percentile() function used to compute the nth percentile of the give

```
In [42]: a
```

```
Out[42]: array([31, 61, 94, 22, 87, 25, 26, 28, 17, 23, 11, 95, 37, 39, 13],
                dtype=int32)
```

```
In [43]: np.percentile(a,31)
```

```
Out[43]: np.float64(23.68)
```

```
In [44]: np.percentile(a,95)
```

```
Out[44]: np.float64(94.3)
```

```
In [ ]: np.histogram
```

Numpy has a built-in numpy.histogram() function which represents the freq

```
In [ ]: ### np.isin
```

With the help of numpy.isin() method, we can see that one array having va
numpy array having different elements with different size

```
In [55]: a
```

```
Out[55]: (array([11,  4]), array([ 0,  50, 100]))
```

```
In [61]: g = np.array([10,30,40,5,6,7,8,91,34,78,40,90])
```

```
In [57]: items = [10,40,30,90]
```

```
In [62]: g[np.isin(g,items)]
```

```
Out[62]: array([10, 30, 40, 40, 90])
```

```
In [ ]: ### np.flip  
The numpy.flip() function reverses the order of array elements along the s
```

```
In [63]: g
```

```
Out[63]: array([10, 30, 40,  5,  6,  7,  8, 91, 34, 78, 40, 90])
```

```
In [64]: np.flip(g)
```

```
Out[64]: array([90, 40, 78, 34, 91,  8,  7,  6,  5, 40, 30, 10])
```

```
In [65]: b
```

```
Out[65]: array([[71, 27, 77, 60],  
               [48, 70, 92, 65],  
               [64, 44, 82, 51],  
               [17, 40, 99, 94],  
               [28, 41, 57, 82],  
               [65, 82, 48, 40]], dtype=int32)
```

```
In [66]: np.flip(b,axis = 1)
```

```
Out[66]: array([[60, 77, 27, 71],  
               [65, 92, 70, 48],  
               [51, 82, 44, 64],  
               [94, 99, 40, 17],  
               [82, 57, 41, 28],  
               [40, 48, 82, 65]], dtype=int32)
```

```
In [ ]: np.put  
  
The numpy.put() function replaces specific elements of an array with give
```

```
In [67]: g
```

```
Out[67]: array([10, 30, 40,  5,  6,  7,  8, 91, 34, 78, 40, 90])
```

```
In [75]: np.put(g,[3,4],[50,60])
```

```
In [73]: np.delete  
The numpy.delete() function returns a new array with the deletion of sub-
```

```
In [74]: g
```

```
Out[74]: array([10, 30, 40, 50, 60,  7,  8, 91, 34, 78, 40, 90])
```

```
In [80]: np.delete(g,[5,6])
```

```
Out[80]: array([10, 30, 40, 50, 60, 91, 34, 78, 40, 90])
```

```
In [81]: g
```

```
Out[81]: array([10, 30, 40, 50, 60, 7, 8, 91, 34, 78, 40, 90])
```

```
In [ ]: ### Set functions
```

```
- np.union1d  
- np.intersect1d  
- np.setdiff1d  
- np.setxor1d  
- np.in1d
```

```
In [84]: m = np.array([1,2,3,4,5])  
n = np.array([3,4,5,6,7])
```

```
In [85]: np.union1d(m,n)
```

```
Out[85]: array([1, 2, 3, 4, 5, 6, 7])
```

```
In [86]: np.intersect1d(m,n)
```

```
Out[86]: array([3, 4, 5])
```

```
In [87]: #Returns elements present in first array but not in second array.  
np.setdiff1d(n,m)
```

```
Out[87]: array([6, 7])
```

```
In [88]: np.setdiff1d(m,n)
```

```
Out[88]: array([1, 2])
```

```
In [ ]: np.setxor1d()
```

Returns elements that are present **in** only one array (**not** common).

```
In [89]: np.setxor1d(m,n)
```

```
Out[89]: array([1, 2, 6, 7])
```

```
In [ ]: np.in1d()
```

Checks whether each element of first array exists **in** second array.

```
In [91]: np.in1d(m,n)
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_11832\207282267.py:1: DeprecationWarning: `in1d` is deprecated. Use `np.isin` instead.
np.in1d(m,n)

```
Out[91]: array([False, False,  True,  True,  True])
```

```
In [ ]: np.clip  
numpy.clip() function is used to Clip (limit) the values in an array
```

```
In [92]: g
```

```
Out[92]: array([10, 30, 40, 50, 60, 7, 8, 91, 34, 78, 40, 90])
```

```
In [96]: np.clip(g,30,78)
```

```
Out[96]: array([30, 30, 40, 50, 60, 30, 30, 78, 34, 78, 40, 78])
```

```
In [ ]:
```